

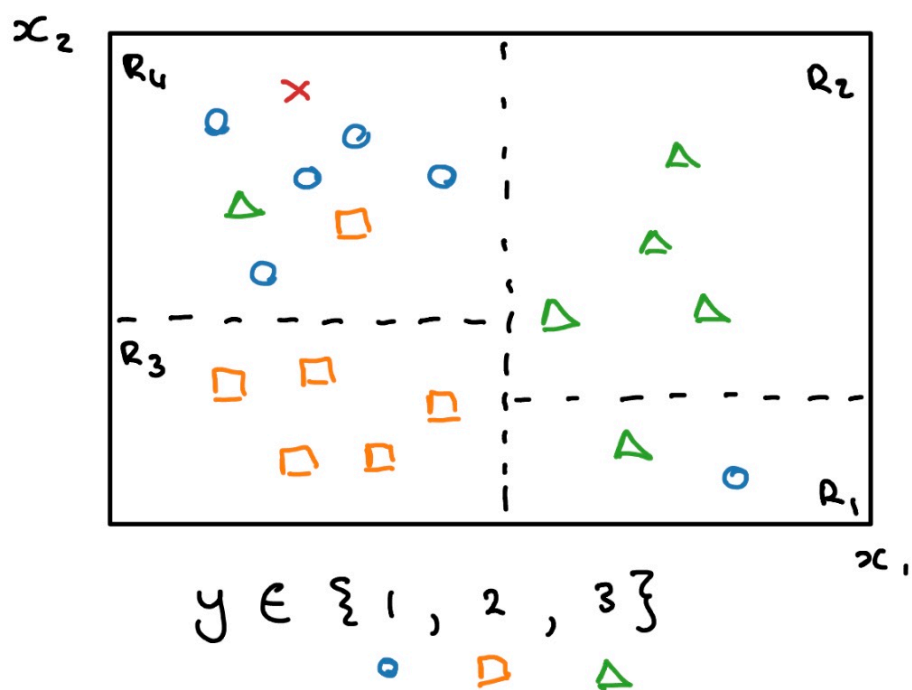
Using decision trees for classification:

Fraction of points in region R_m from class k :

$$\hat{p}_{m,k} = \frac{\# \text{points in } R_m \text{ with label } k}{\# \text{points in } R_m}$$

$$= \frac{1}{N_m} \sum_{i: x^{(i)} \in R_m} \mathbb{I} \{y^{(i)} = k\}$$

$$\hat{y}_m = \arg \max_k \hat{p}_{m,k}$$



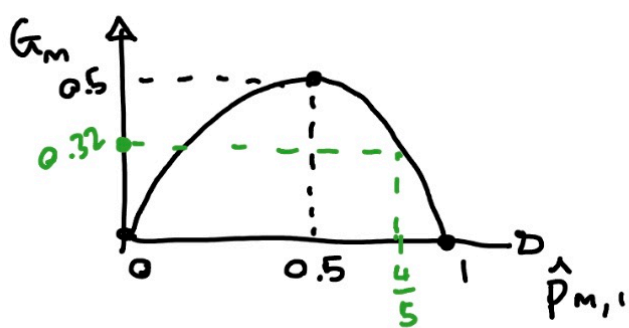
Tree growing algorithm:

1. Start at top of tree (everything in one region) top-down
2. for each leaf node (region):
for each feature x_j and split point s :
Calculate reduction in leaf impurity
3. Choose best (j, s) combination to split;
create new child nodes (regions) greedy
4. Repeat from (2) until stop condition is met recursive

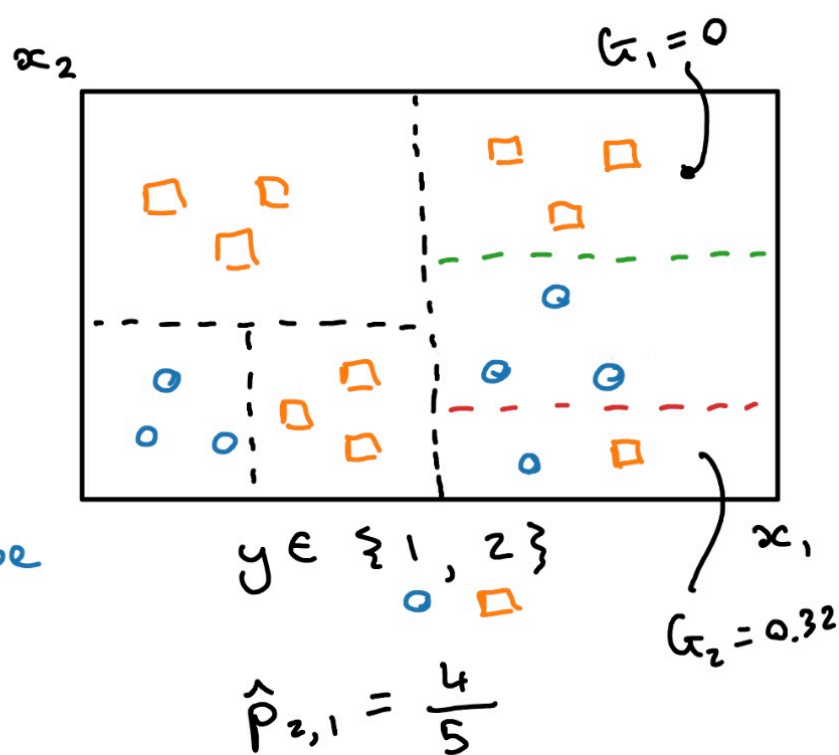
Gini index: $G_m = \sum_{k=1}^K \hat{p}_{m,k} (1 - \hat{p}_{m,k})$

For two classes: $\hat{p}_{m,1}$ & $\hat{p}_{m,2}$

$$\hat{p}_{m,2} = 1 - \hat{p}_{m,1}$$



$$\text{Entropy: } - \sum_{k=1}^K \hat{p}_{m,k} \log_2(\hat{p}_{m,k})$$



Example practice:

For $k = \text{No}$:

For $k = \text{Yes}$:

$$\hat{p}_{\text{No}} = \frac{3}{6}$$

$$= 0.5$$

$$\hat{p}_{\text{Yes}} = \frac{3}{6}$$

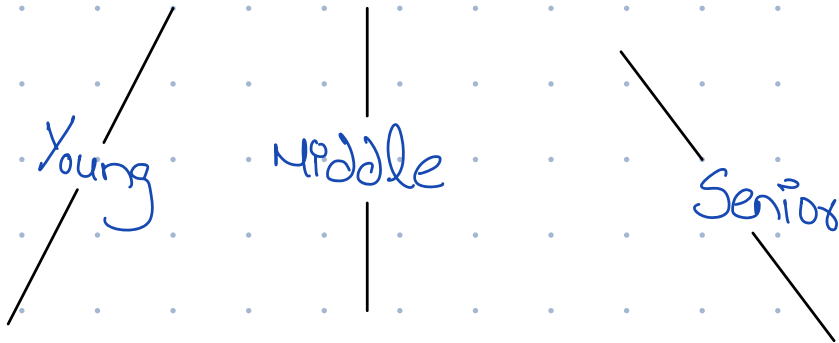
$$= 0.5$$

$$\text{Entropy: } - \frac{3}{6} \log_2\left(\frac{3}{6}\right) - \frac{3}{6} \log_2\left(\frac{3}{6}\right)$$

ID	Age	Income	Buys? (Target)
1	Young	High	No
2	Young	Low	No
3	Middle	High	Yes
4	Middle	Low	Yes
5	Senior	High	Yes
6	Senior	Low	No

Select first split:

Age
Samples: 6
Entropy: 1
class: ?



Samples: 2
Entropy: 0
class: No

Samples: 2
Entropy: 0
class: Yes

Samples: 2
Entropy: 1
class: ?



$$P_{\text{No}} = \frac{2}{2} = 1$$



$$P_{\text{Yes}} = \frac{2}{2} = 1$$



$$P_{\text{No}} = \frac{1}{2}$$

$$P_{\text{Yes}} = \frac{1}{2}$$

$$\begin{aligned} \text{Weighted Entropy} &= \frac{2}{6} (0) + \frac{2}{6} (0) + \frac{2}{6} (1) \\ &= 0.34 \end{aligned}$$

$$\begin{aligned} \text{Information gain} &= 1 - 0.34 \\ &= 0.66 \end{aligned}$$

Income
Samples: 6
Entropy: 1
class: ?

High

Low

Samples: 3
Entropy: 0.9
class: Yes

Samples: 3
Entropy:
class:

$$\hat{P}_{\text{yes}} = \frac{2}{3}$$

$$\hat{P}_{\text{no}} = \frac{1}{3}$$

$$\hat{P}_{\text{yes}} = \frac{1}{3}$$

$$\hat{P}_{\text{no}} = \frac{2}{3}$$

$$\text{Entropy} = -\frac{2}{3} \log_2\left(\frac{2}{3}\right) - \frac{1}{3} \log_2\left(\frac{1}{3}\right)$$

$$\text{Entropy} = 0.91$$

$$= 0.91$$

$$\text{Weighted Entropy} = \frac{3}{6} (0.91) + \frac{3}{6} (0.91)$$

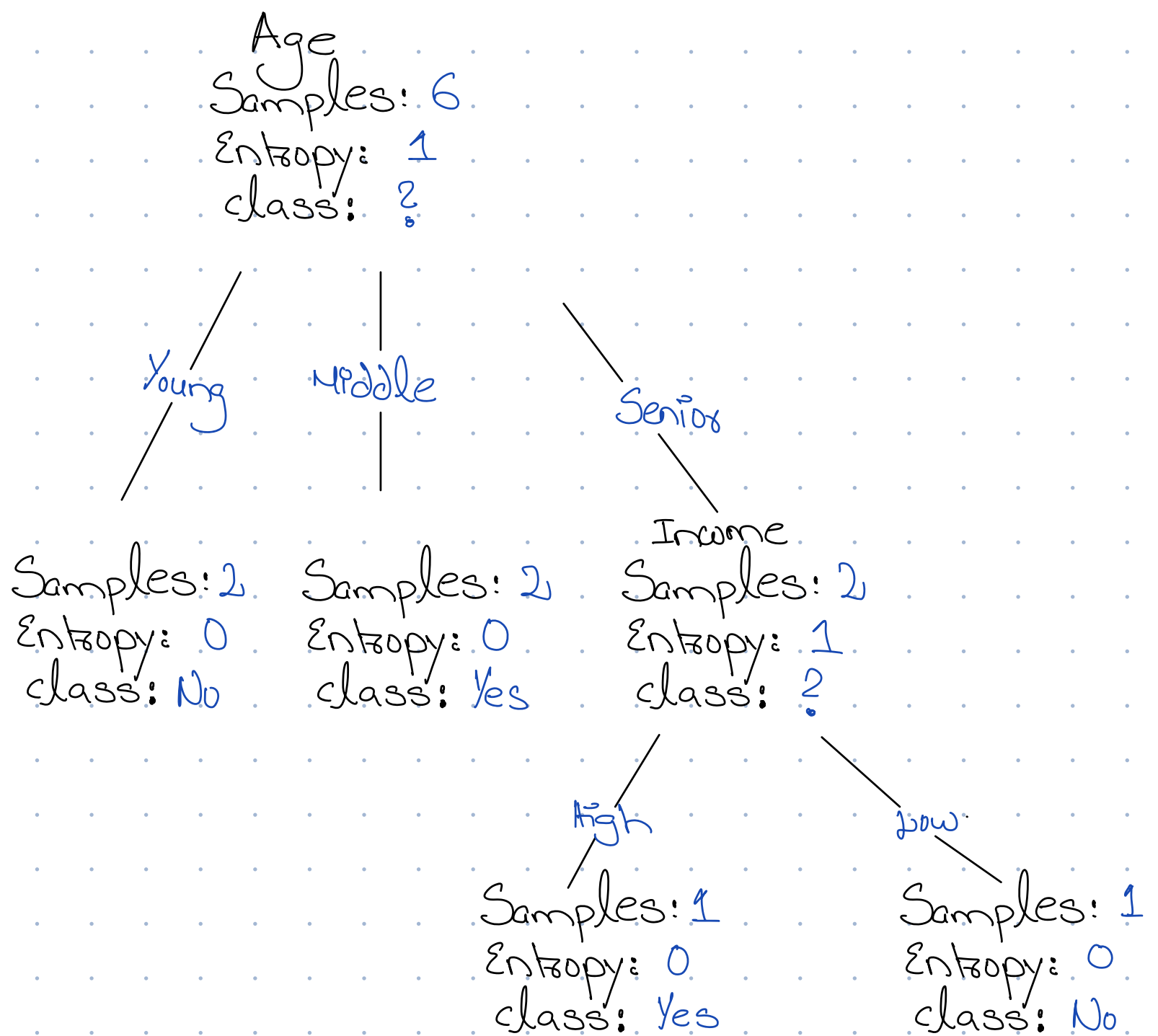
$$= 0.91$$

$$\text{Information Gain} = 1 - 0.91$$

$$= 0.09$$

So, we choose Age

Select second split:



$$\text{Information gain} = 1 - 0 = 1$$

Test:

Test A: Age = Middle, Income = Low

, Buys? Yes

Test B: Age = Senior, Income = Low

, Buys? No

Test C: Age = Senior, Income = High

, Buys? Yes

NOTE:

If feature A is categorical & if I chose to split at feature A, then I can't split using feature A.

If feature A is continuous & if I choose to split at thres_1 then

I can split at feature A again but with a different thres_2 i.e. $\text{thres}_2 \neq \text{thres}_1$.

Regression tree model

Model: $f(x; \theta) = \sum_{m=1}^M c_m I\{x \in R_m\}$

Loss: $J(\theta) = \sum_{n=1}^N (y^{(n)} - f(x^{(n)}))^2$
 $= \sum_{n=1}^N (y^{(n)} - \sum_{m=1}^M c_m I\{x^{(n)} \in R_m\})^2$

Parameters: $\left. \begin{matrix} R_1, \dots, R_M \\ c_1, \dots, c_M \end{matrix} \right\} \theta$

If we knew regions (but not the c 's):

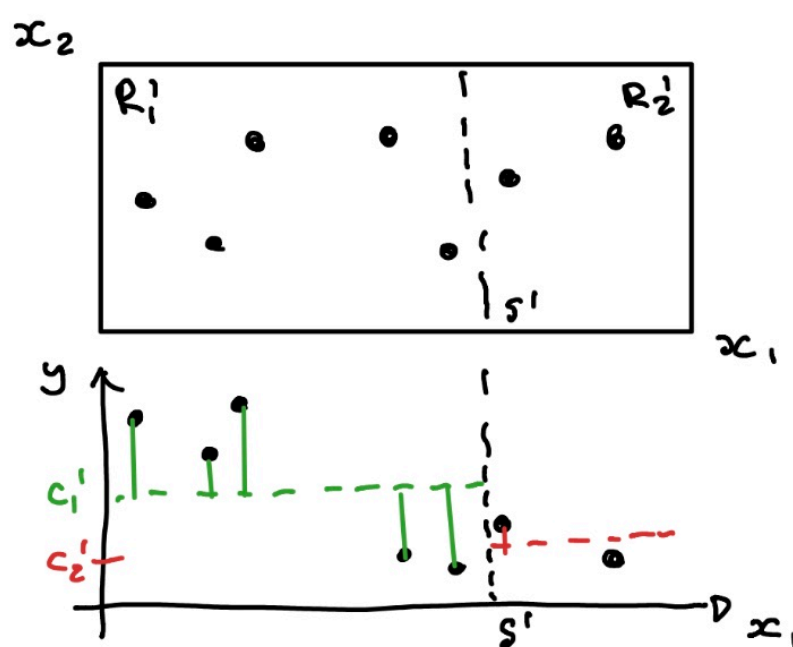
$$\Rightarrow c_m = \frac{1}{N_m} \sum_{i: x^{(i)} \in R_m} y^{(i)}$$

How do we learn θ ?

training items in R_m

Tree growing algorithm:

1. Start at top of tree (everything in one region)
2. for each leaf node (region):
 for each feature x_j and split point s :
 Calculate reduction in loss if we split there
3. Choose best (j, s) combination to split;
 create new child nodes (regions)
4. Repeat from (2) until stop condition is met



$$\min_{j, s} \left\{ \sum_{i: x^{(i)} \in R_1} (y^{(i)} - c_1)^2 + \sum_{i: x^{(i)} \in R_2} (y^{(i)} - c_2)^2 \right\}$$

Example,

$$C = \frac{30 + 35 + 70 + 80}{5}$$

$$= 43$$

$$\text{MSE} = \frac{(30 - 43)^2 + (35 - 43)^2 + (70 - 43)^2 + (80 - 43)^2}{4}$$

$$= 169$$

ID	Experience (X)	Salary (Y)
1	1	30
2	2	35
3	4	70
4	5	80

Experience ≤ 3

Samples: 4

C: 43

MSE: 169

True

False

Samples: 2

C: 32.5

MSE: 6.25



$$C = \frac{30 + 35}{2}$$

$$= 32.5$$

$$MSE = \frac{(30 - 32.5)^2 + (35 - 32.5)^2}{2}$$

$$= 6.25$$

Samples: 2

C: 75

MSE: 25



$$C = \frac{70 + 80}{2}$$

$$= 75$$

$$MSE = \frac{(70 - 75)^2 + (80 - 75)^2}{2}$$

$$= 25$$

$$\text{Total} = (6.25 + 25)$$

$$= 31.25$$

Lets choose a separate threshold

Experience ≤ 4

Samples: 4

C: 43

MSE: 169

True

False

ID	Experience (X)	Salary (Y)
1	1	30
2	2	35
3	4	70
4	5	80

Samples: 3

C: 45

MSE: 316.66



$$C = \frac{30 + 35 + 70}{3}$$

$$= 45$$

$$\begin{aligned} \text{MSE} &= \frac{(30 - 45)^2 + (35 - 45)^2 + (70 - 45)^2}{3} \\ &= 316.66 \end{aligned}$$

Samples: 1

C: 80

MSE: 0



$$C = \frac{80}{1}$$

$$= 80$$

$$\begin{aligned} \text{MSE} &= \frac{(80 - 80)^2}{1} \\ &= 0 \end{aligned}$$

$$\begin{aligned} \text{Total} &= 316.66 + 0 \\ &= 316.66 \end{aligned}$$

So, we choose Experience ≤ 3

↳ we try different threshold
I choose one with lowest
total MSE

Bagging

Go out & create B datasets

Bagging:

$$\underline{X} = \begin{bmatrix} - & (\underline{x}^{(1)})^T & - \\ - & (\underline{x}^{(2)})^T & - \\ & \vdots & \\ - & (\underline{x}^{(n)})^T & - \end{bmatrix}$$

$$\underline{y} = \begin{bmatrix} y^{(1)} \\ y^{(2)} \\ \vdots \\ y^{(n)} \end{bmatrix}$$

$$\begin{matrix} \updownarrow \\ \updownarrow \end{matrix} \begin{bmatrix} \underline{x}_1 \\ \vdots \\ \underline{x}_B \end{bmatrix}; \begin{bmatrix} y_1 \\ \vdots \\ y_B \end{bmatrix}$$

$$\Rightarrow f_1(\underline{x}; \underline{\Theta}_1)$$

$$\Rightarrow f_2(\underline{x}; \underline{\Theta}_2)$$

$\vdots$

$$\Rightarrow f_B(\underline{x}; \underline{\Theta}_B)$$

In practice, we sample with replacement from X to create B datasets.

$$\underline{X} = \begin{bmatrix} \text{orange} \\ \text{blue} \\ \text{green} \end{bmatrix}; \underline{y} = \begin{bmatrix} \text{orange} \\ \text{blue} \\ \text{green} \end{bmatrix}$$

$$\begin{matrix} \updownarrow \\ \updownarrow \end{matrix} \begin{bmatrix} \underline{x}_1 \\ \vdots \\ \underline{x}_2 \end{bmatrix}; \begin{bmatrix} y_1 \\ \vdots \\ y_2 \end{bmatrix}$$

$$\Rightarrow f_1(\underline{x}; \underline{\Theta}_1)$$

$$f(\underline{x}; \underline{\Theta}) = \frac{1}{B} \sum_{b=1}^B f_b(\underline{x}; \underline{\Theta}_b)$$

↑ Regression

Classification: Majority or weighted voting

$$\begin{matrix} \updownarrow \\ \updownarrow \end{matrix} \begin{bmatrix} \underline{x}_2 \\ \vdots \end{bmatrix}; \begin{bmatrix} y_2 \\ \vdots \end{bmatrix}$$

$$\Rightarrow f_2(\underline{x}; \underline{\Theta}_2)$$

$\vdots$

$$f_B(\underline{x}; \underline{\Theta}_B)$$

Now,

we simply train a separate or same model on each of created datasets.

Our final prediction is average prediction from each model

$$f(\underline{x}; \underline{\Theta}) = \frac{1}{B} \sum_{b=1}^B f_b(\underline{x}; \underline{\Theta}_b)$$

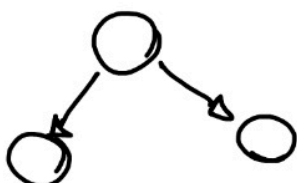
Random Forest

Random forests:

- Specifically used with decision and regression trees.
- Use bagging: Train each tree on different bootstrap sample. But then also...
- Every time we split, only consider $M < D$ random features
- $M = \sqrt{D}$ is often used

For Regression:

$$f(\underline{x}; \underline{\theta}) = \frac{1}{B} \sum_{b=1}^B f_b(\underline{x}; \underline{\theta}_b)$$



For classification:

- Take majority voting
 - Further, we can weight each model's prediction by purity of leaf node
- $\hat{p}_{m,k}$

Boosting for Regression:

1. Initialise $r^{(n)} = y^{(n)}$ for all N training items and set $f(\mathbf{x}; \boldsymbol{\theta}) = 0$.

2. for iteration $b = 1$ to B :

(a) Fit model $f_b(\mathbf{x}; \boldsymbol{\theta}_b)$ to inputs $\mathbf{X}$, outputs $\mathbf{r}$.

(b) Update model by adding shrunken version:

$$f(\mathbf{x}; \boldsymbol{\theta}) \leftarrow f(\mathbf{x}; \boldsymbol{\theta}) + \lambda f_b(\mathbf{x}; \boldsymbol{\theta}_b)$$

(c) Update the residuals:

$$r^{(n)} \leftarrow r^{(n)} - \lambda f_b(\mathbf{x}; \boldsymbol{\theta}_b)$$

3. Final model: $f(\mathbf{x}; \boldsymbol{\theta}) = \sum_{b=1}^B \lambda f_b(\mathbf{x}; \boldsymbol{\theta}_b)$

This is gradient of MSE loss w.r.t to function ($z = wx + b$ in logistic regression). So, this is called gradient boosting

$$\underline{X} = \begin{bmatrix} -(\underline{x}^{(1)})^T \\ -(\underline{x}^{(2)})^T \\ \vdots \\ -(\underline{x}^{(N)})^T \end{bmatrix}; \quad \underline{r} = \begin{bmatrix} r^{(1)} \\ r^{(2)} \\ \vdots \\ r^{(N)} \end{bmatrix}$$

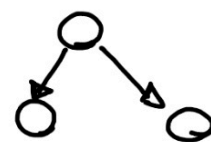
At $b = 1$:

$\underline{r} = \underline{y}$, so we are just fitting a model to inputs $\underline{X}$, outputs $\underline{y}$

At $b > 1$:

Fitting a model to the residuals.

Weak learner:



Decision tree stub (only one split)

1. Initialise $r^{(n)} = y^{(n)}$ for all N training items and set $f(\mathbf{x}; \boldsymbol{\theta}) = 0$.

At $b = 1$: $\underline{r} = \underline{y}$

2. for iteration $b = 1$ to B :

(a) Fit model $f_b(\mathbf{x}; \boldsymbol{\theta}_b)$ to inputs $\mathbf{X}$, outputs $\mathbf{r}$.

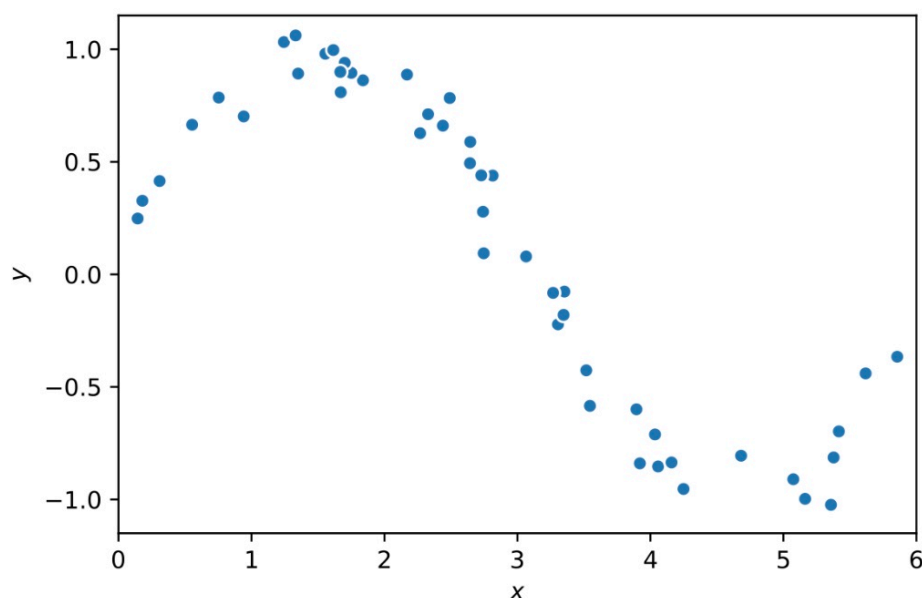
(b) Update model by adding shrunken version:

$$f(\mathbf{x}; \boldsymbol{\theta}) \leftarrow f(\mathbf{x}; \boldsymbol{\theta}) + \lambda f_b(\mathbf{x}; \boldsymbol{\theta}_b)$$

(c) Update the residuals:

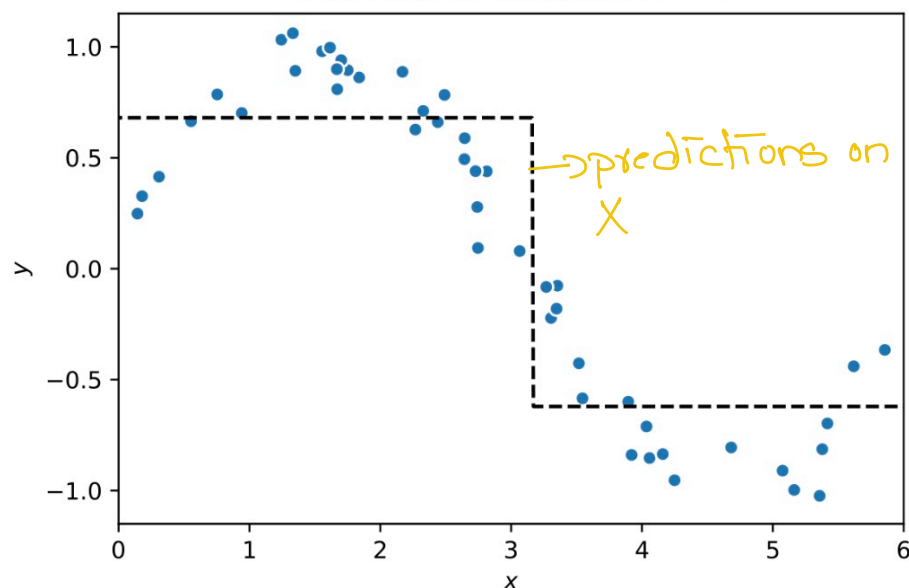
$$r^{(n)} \leftarrow r^{(n)} - \lambda f_b(\mathbf{x}; \boldsymbol{\theta}_b)$$

3. Final model: $f(\mathbf{x}; \boldsymbol{\theta}) = \sum_{b=1}^B \lambda f_b(\mathbf{x}; \boldsymbol{\theta}_b)$



$f_1(\mathbf{x}; \boldsymbol{\theta}_1)$

Model trained at iteration $b = 1$



1. Initialise $r^{(n)} = y^{(n)}$ for all N training items and set $f(\mathbf{x}; \boldsymbol{\theta}) = 0$.

2. for iteration $b = 1$ to B :

(a) Fit model $f_b(\mathbf{x}; \boldsymbol{\theta}_b)$ to inputs $\mathbf{X}$, outputs $\mathbf{r}$.

(b) Update model by adding shrunken version:

$$f(\mathbf{x}; \boldsymbol{\theta}) \leftarrow f(\mathbf{x}; \boldsymbol{\theta}) + \lambda f_b(\mathbf{x}; \boldsymbol{\theta}_b)$$

(c) Update the residuals:

$$r^{(n)} \leftarrow r^{(n)} - \lambda f_b(\mathbf{x}; \boldsymbol{\theta}_b)$$

3. Final model: $f(\mathbf{x}; \boldsymbol{\theta}) = \sum_{b=1}^B \lambda f_b(\mathbf{x}; \boldsymbol{\theta}_b)$

1. Initialise $r^{(n)} = y^{(n)}$ for all N training items and set $f(\mathbf{x}; \boldsymbol{\theta}) = 0$.

2. for iteration $b = 1$ to B :

(a) Fit model $f_b(\mathbf{x}; \boldsymbol{\theta}_b)$ to inputs $\mathbf{X}$, outputs $\mathbf{r}$.

(b) Update model by adding shrunk version:

$$f(\mathbf{x}; \boldsymbol{\theta}) \leftarrow f(\mathbf{x}; \boldsymbol{\theta}) + \lambda f_b(\mathbf{x}; \boldsymbol{\theta}_b)$$

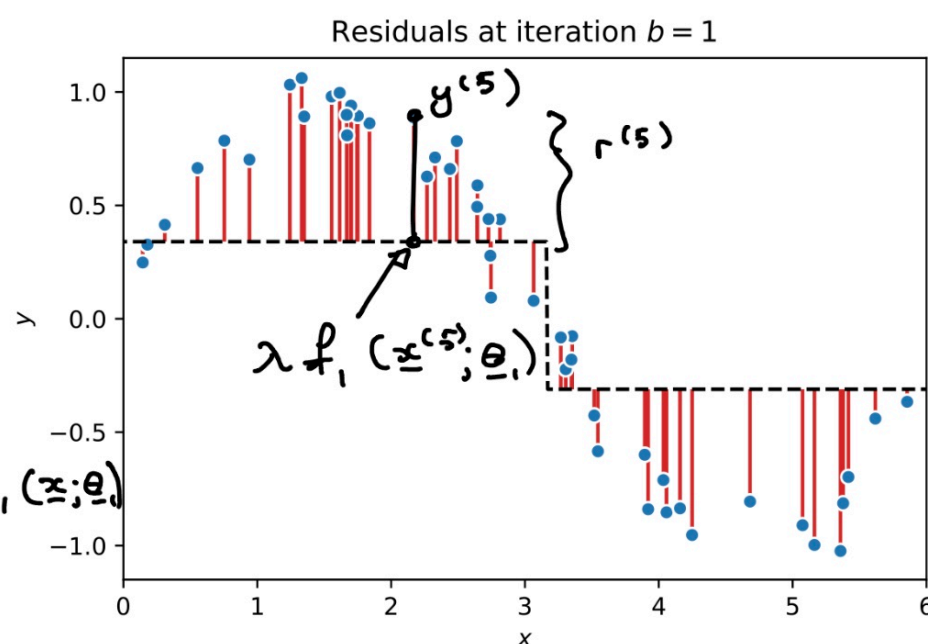
(c) Update the residuals:

$$r^{(n)} \leftarrow r^{(n)} - \lambda f_b(\mathbf{x}; \boldsymbol{\theta}_b)$$

At $b=1$:

$$r^{(n)} \leftarrow y^{(n)} - \lambda f_1(\mathbf{x}; \boldsymbol{\theta}_1)$$

3. Final model: $f(\mathbf{x}; \boldsymbol{\theta}) = \sum_{b=1}^B \lambda f_b(\mathbf{x}; \boldsymbol{\theta}_b)$



$$f_2(\mathbf{x}; \boldsymbol{\theta}_2)$$

1. Initialise $r^{(n)} = y^{(n)}$ for all N training items and set $f(\mathbf{x}; \boldsymbol{\theta}) = 0$.

2. for iteration $b = 1$ to B :

$b=2$

(a) Fit model $f_b(\mathbf{x}; \boldsymbol{\theta}_b)$ to inputs $\mathbf{X}$, outputs $\mathbf{r}$.

(b) Update model by adding shrunk version:

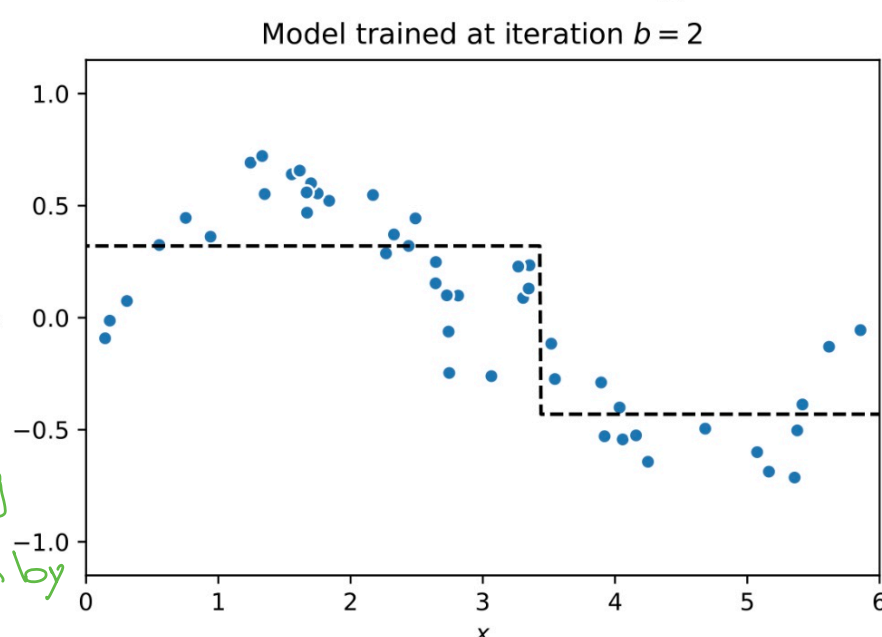
$$f(\mathbf{x}; \boldsymbol{\theta}) \leftarrow f(\mathbf{x}; \boldsymbol{\theta}) + \lambda f_b(\mathbf{x}; \boldsymbol{\theta}_b)$$

(c) Update the residuals:

$$r^{(n)} \leftarrow r^{(n)} - \lambda f_b(\mathbf{x}; \boldsymbol{\theta}_b)$$

After $b=1$, we are essentially adding corrections to the original prediction by $b=1$.

3. Final model: $f(\mathbf{x}; \boldsymbol{\theta}) = \sum_{b=1}^B \lambda f_b(\mathbf{x}; \boldsymbol{\theta}_b)$



You are 100% correct that for $b > 1$, the model predicts a small residual (e.g., "add 2.5"). If that was your **final** answer, the prediction would be terrible.

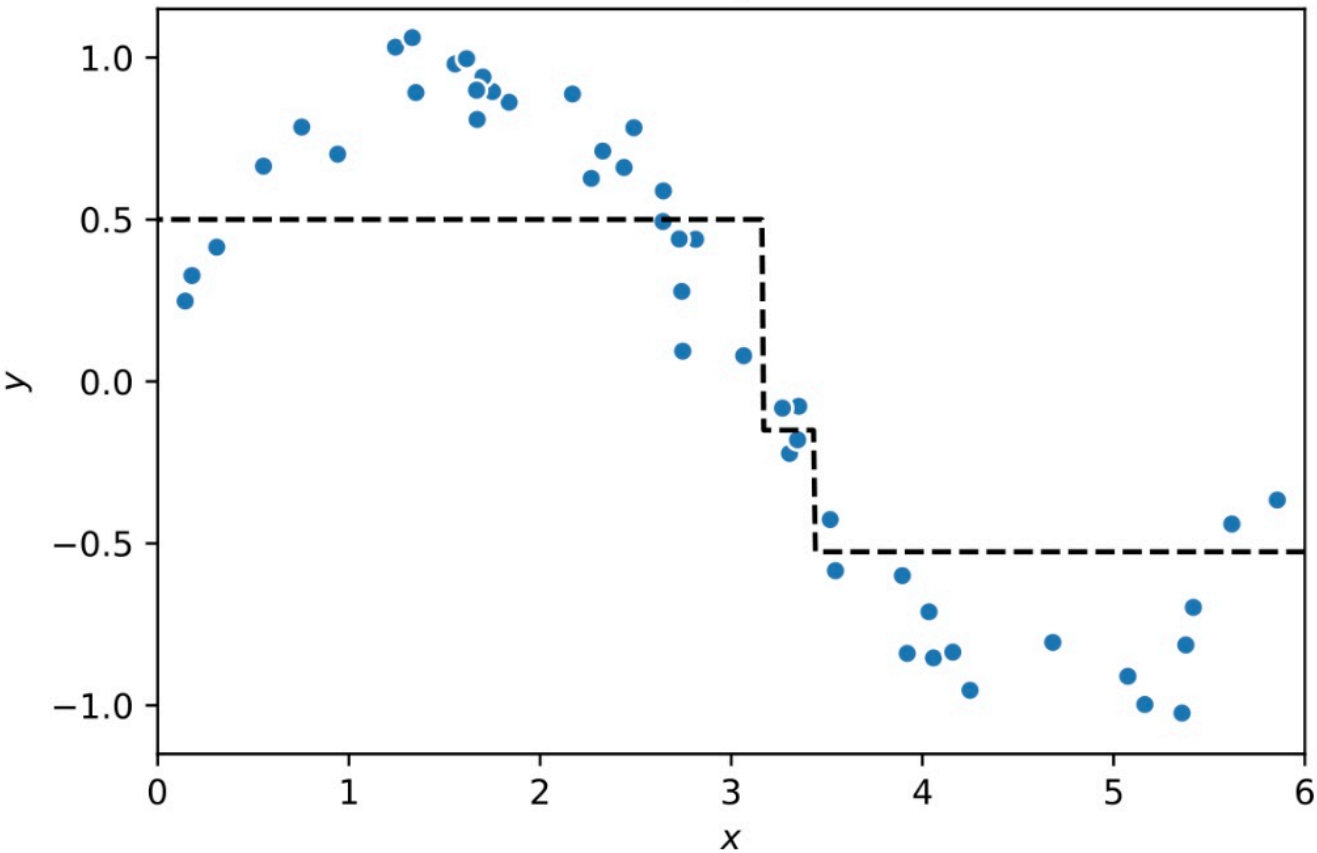
The secret is **Step 3** in your image:

$$f(x; \theta) = \sum_{b=1}^B \lambda f_b(x; \theta_b)$$

The final prediction is the **SUM** of all those small residual predictions. You aren't replacing your old answer; you are **adding a correction** to it.

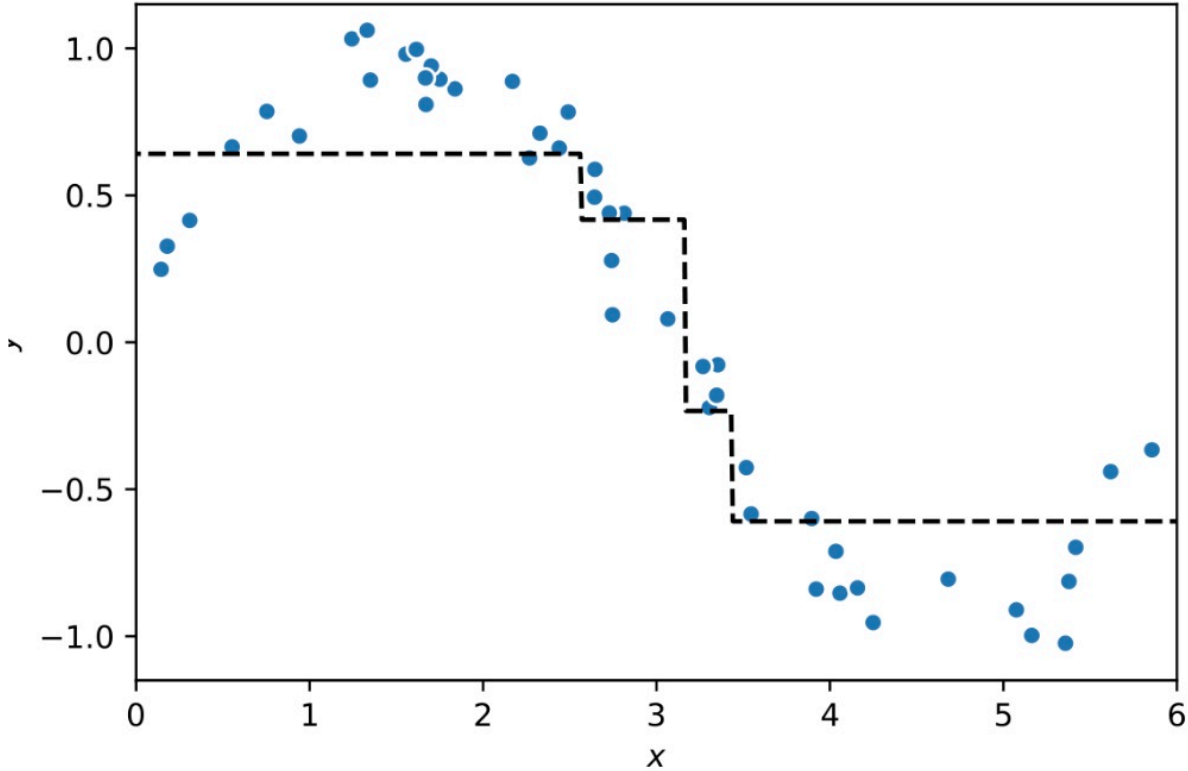
$$\lambda f_2(x; \underline{\theta}_2) + \lambda f_1(x; \underline{\theta}_1)$$

Combined model output at iteration $b = 2$

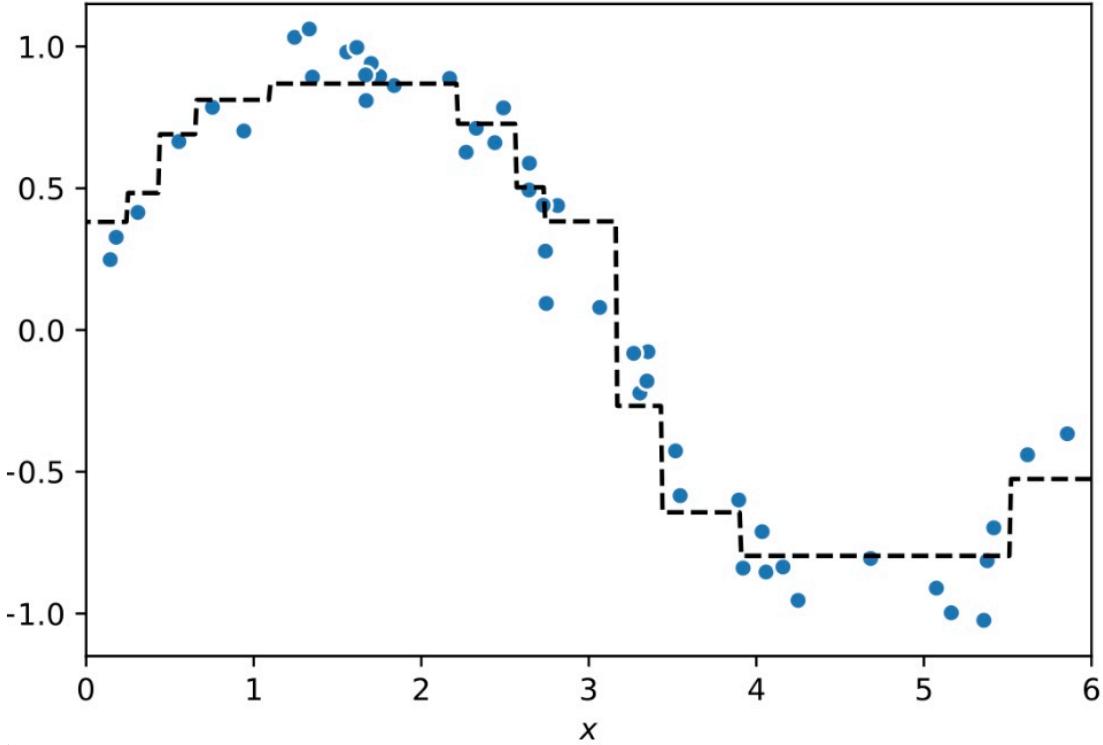


$$\lambda f_3(x; \underline{\theta}_3) + \lambda f_2(x; \underline{\theta}_2) + \lambda f_1(x; \underline{\theta}_1)$$

Combined model output at iteration $b = 3$



Combined model output at iteration $b = 15$



AdaBoosting: boosting for classification

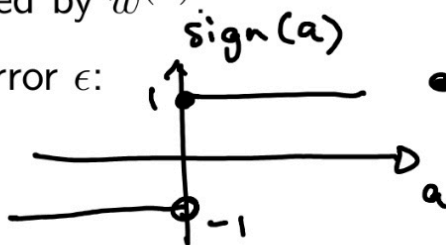
1. Initialise training item weights $w^{(n)} = \frac{1}{N}$ for all N training items.

2. for iteration $b = 1$ to B :

(a) Fit model $f_b(\mathbf{x}; \theta_b)$ so that it minimises classification error weighted by $w^{(n)}$

(b) Set model weight using error ϵ :

$$\lambda_b = \frac{1}{2} \log \left(\frac{1 - \epsilon}{\epsilon} \right)$$



(c) Update training item weights:

$w^{(n)} \leftarrow w^{(n)} e^{-\lambda_b}$ if $f_b(\mathbf{x}^{(n)}; \theta_b)$ correct

$w^{(n)} \leftarrow w^{(n)} e^{\lambda_b}$ if $f_b(\mathbf{x}^{(n)}; \theta_b)$ incorrect

3. Final model: $f(\mathbf{x}; \theta) = \text{sign} \left[\sum_{b=1}^B \lambda_b f_b(\mathbf{x}; \theta_b) \right]$

Setting:

- Binary classification: $y \in \{-1, 1\}$

- Going to train B models, each $f_b(\mathbf{x}; \theta_b) \in \{-1, 1\}$

- Going to combine weighted votes:

$$f(\mathbf{x}; \theta) = \text{sign} \left[\sum_{b=1}^B \lambda_b f_b(\mathbf{x}; \theta_b) \right]$$

$$\begin{matrix} w^{(1)} & -(\mathbf{x}^{(1)})^T & - & y^{(1)} \\ w^{(2)} & -(\mathbf{x}^{(2)})^T & - & y^{(2)} \\ \vdots & \vdots & & \vdots \\ w^{(N)} & & & \end{matrix}$$

Intuitively,

we will fit a classifier to data & check how big & on what example our model messed up on.

So, we adjust w_i & in next iteration we penalize the model more if it mis-classifies the same example.

1. Initialise training item weights $w^{(n)} = 1$ for all N training items.

2. for iteration $b = 1$ to B :

(a) Fit model $f_b(\mathbf{x}; \theta_b)$ so that it minimises classification error weighted by $w^{(n)}$.

(b) Set model weight using error ϵ :

$$\lambda_b = \frac{1}{2} \log \left(\frac{1 - \epsilon}{\epsilon} \right) \quad \text{--- (1)}$$

(c) Update training item weights:

$w^{(n)} \leftarrow w^{(n)} e^{-\lambda_b}$ if $f_b(\mathbf{x}^{(n)}; \theta_b)$ correct

$w^{(n)} \leftarrow w^{(n)} e^{\lambda_b}$ if $f_b(\mathbf{x}^{(n)}; \theta_b)$ incorrect (2)

3. Final model: $f(\mathbf{x}; \theta) = \text{sign} \left[\sum_{b=1}^B \lambda_b f_b(\mathbf{x}; \theta_b) \right]$

(a) Minimise $\sum_{n=1}^N w^{(n)} \mathbb{I} \{ y^{(n)} \neq f_b(\mathbf{x}^{(n)}; \theta_b) \}$

$$(b) \epsilon = \frac{\sum_{n=1}^N w^{(n)} \mathbb{I} \{ y^{(n)} \neq f_b(\mathbf{x}^{(n)}; \theta_b) \}}{\sum_{n=1}^N w^{(n)}}$$

If $\epsilon \rightarrow 0$ (very good classifier)

then λ_b big

If $\epsilon \rightarrow \frac{1}{2}$ (bad / random classifier)

then $\lambda_b \rightarrow 0$

(c) Correct: $w^{(n)} \leftarrow w^{(n)} \sqrt{\frac{\epsilon}{1 - \epsilon}}$ is close to 0.50, w is small

Incorrect: $w^{(n)} \leftarrow w^{(n)} \sqrt{\frac{1 - \epsilon}{\epsilon}}$ is close to 0, so, w is large

	Normal Decision Tree	AdaBoost Decision Tree
How we count	Each row = 1 vote	Each row = $w^{(n)}$ votes
Probability (p_k)	$p_k = \frac{\text{Count of Class } k}{\text{Total Rows}}$	$p_k = \frac{\text{Sum of Weights for Class } k}{\text{Total Weight}}$
The Goal	Minimize Impurity of Counts	Minimize Impurity of Weights

Feature	Traditional Decision Tree	AdaBoost Stump
Weighting Factor	$\frac{n_j}{n_{parent}}$	$\frac{W_j}{W_{parent}}$

Dry Run:

ID	Income	Age	Location	Buy Car (y)	Weight (w_1)
1	50,000	25	Urban	-1 (No)	0.2
2	60,000	35	Suburban	+1 (Yes)	0.2
3	45,000	23	Rural	-1 (No)	0.2
4	80,000	45	Suburban	+1 (Yes)	0.2
5	70,000	40	Urban	+1 (Yes)	0.2

Parent

Samples = 5
 Entropy = 0.971

$P_{yes} = \frac{0.6}{1}$ $P_{no} = \frac{0.4}{1}$

Entropy = 0.971

Split at Location = Suburban?

Samples = 2

Entropy = 0
class 1

$$P_{yes} = \frac{0.4}{0.4} = 1$$

$$\text{Weighted } E = \frac{2}{5}(0) + \frac{3}{5}(0.918)$$

$$= 0.551$$

$$Ih = 0.42$$

Test each sample using this stump & update weights

$$G_1 = \frac{0.2}{1}$$

This stump predicts No for sample 5 when it is true.

$$\lambda_1 / \alpha_1 = \frac{1}{2} \ln \left(\frac{1-0.2}{0.2} \right)$$

$$= 0.693$$

For sample 1...4 (correct)

$$w_2^1 = w_1^1 e^{-\lambda_1}$$

$$= 0.2 e^{-0.693}$$

$$= 0.1$$

For sample 5 (incorrect)

$$w_2^5 = w_1^5 e^{\lambda_1}$$

$$= 0.2 e^{0.693}$$

$$= 0.4$$

$$\text{Now } W_2^1 = [0.1 \quad 0.1 \quad 0.1 \quad 0.1 \quad 0.4]$$

Normalize:

NOTE:

we have to do this for Age & Income & choose the one that gives highest Ih .

$$N_2 = [0.125 \quad 0.125 \quad 0.125 \quad 0.125 \quad 0.5]$$